

# CS316: ALGORITHMS

## Lecture 3

### ANALYSIS OF NON-RECURSIVE ALGORITHMS

Prepared by: Assoc. Prof. Ensaf Hussein

Presented by:

**Dr. Mohammed El-Said**

**Dr. Ahmed Badawy**

1

- Analyzing Non-recursive Algorithms.

2

- Insertion Sort.

3

- Merge Sort

4

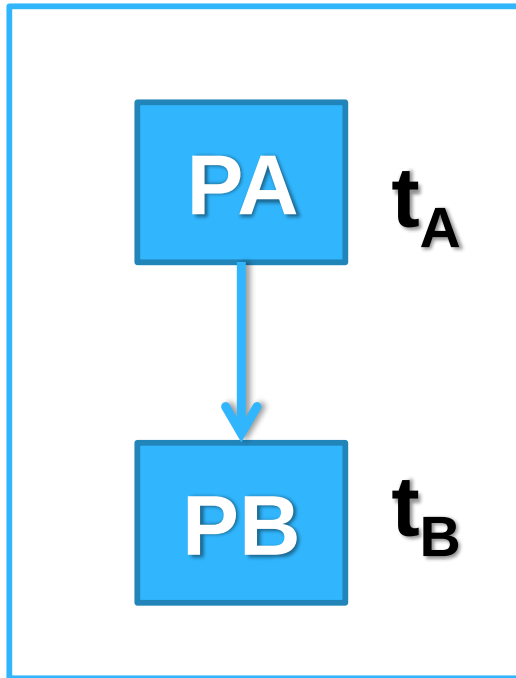
- Appendix : useful formulas for Analysis of Algorithms.

# ANALYZING NON-RECURSIVE ALGORITHMS

# ANALYSIS ALGORITHM CONTROL STRUCTURES

- ☐ Sequencing
- ☐ If-then-else
- ☐ For loop
- ☐ While loop
- ☐ Recursion

# SEQUENCING



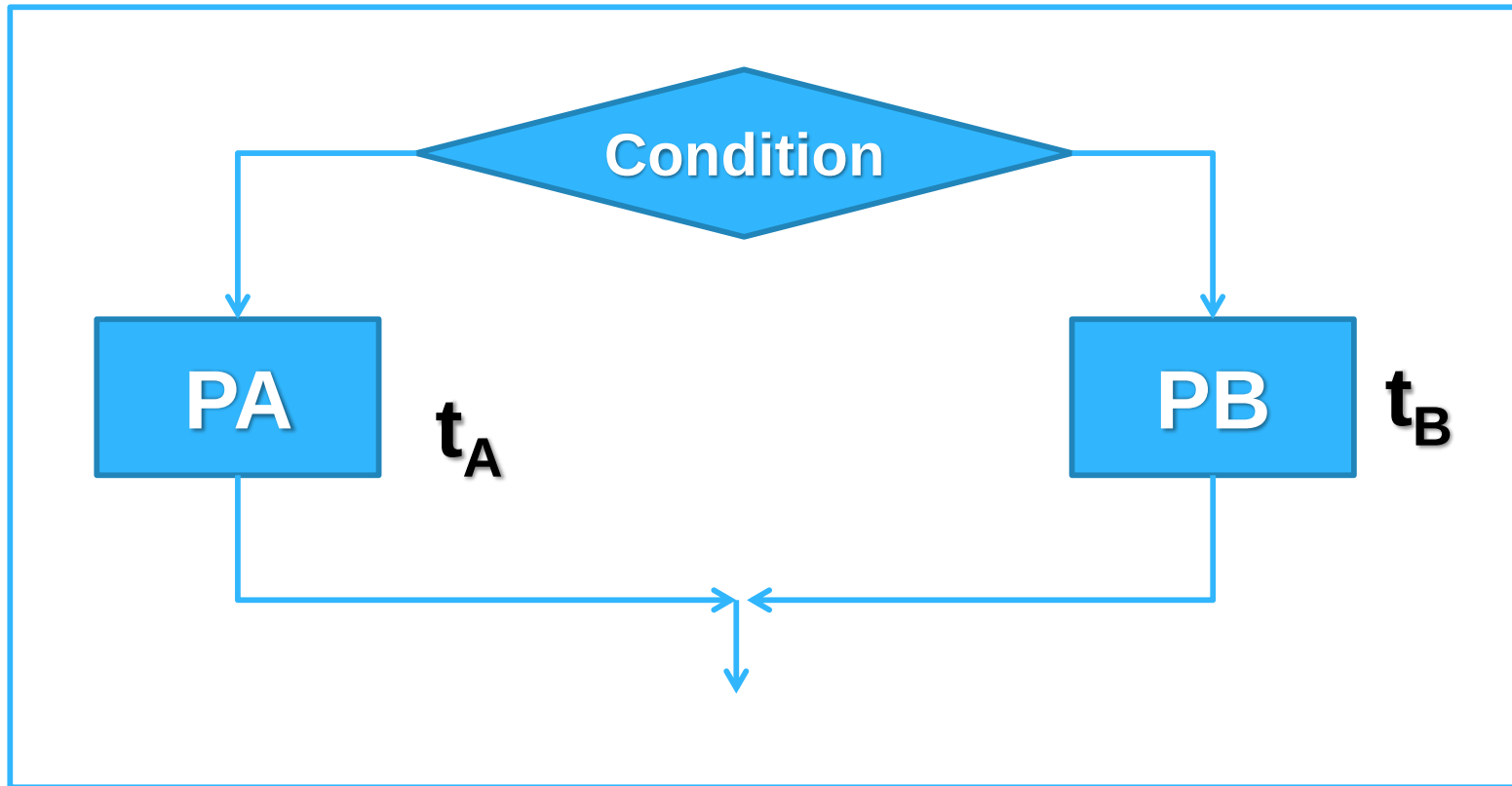
$$t_A + t_B = \max(t_A, t_B)$$

**EX:**

If  $t_A = \theta(n)$  ,  $t_B = \theta(n^2)$

$$\rightarrow t_A + t_B = \max(t_A, t_B) \\ = \theta(n^2)$$

# IF-THEN-ELSE



$$t_A/t_B = \max(t_A, t_B)$$

# FOR LOOP

```
for i ← 1 to n
{
    P(i)
}
```

$$\sum_{i=1}^n t_i$$

```
for i ← 1 to n
{
    for j ← 1 to n
    {
        P(ij)
    }
}
```

$$\sum_{i=1}^n \sum_{j=1}^n t_{ij}$$

# WHILE LOOP

```
While (n>0)
{
  -----
  -----

  Set n ← n-1
}
```

=  $\theta(n)$

```
While (n>0)
{
  -----
  -----

  Set n ← n/2
}
```

=  $\theta(\log n)$

```
Set x ← 0
Set y ← n
While (x<y)
{
  -----
  -----

  Set x ← x+1
  Set y ← y-1
}
```

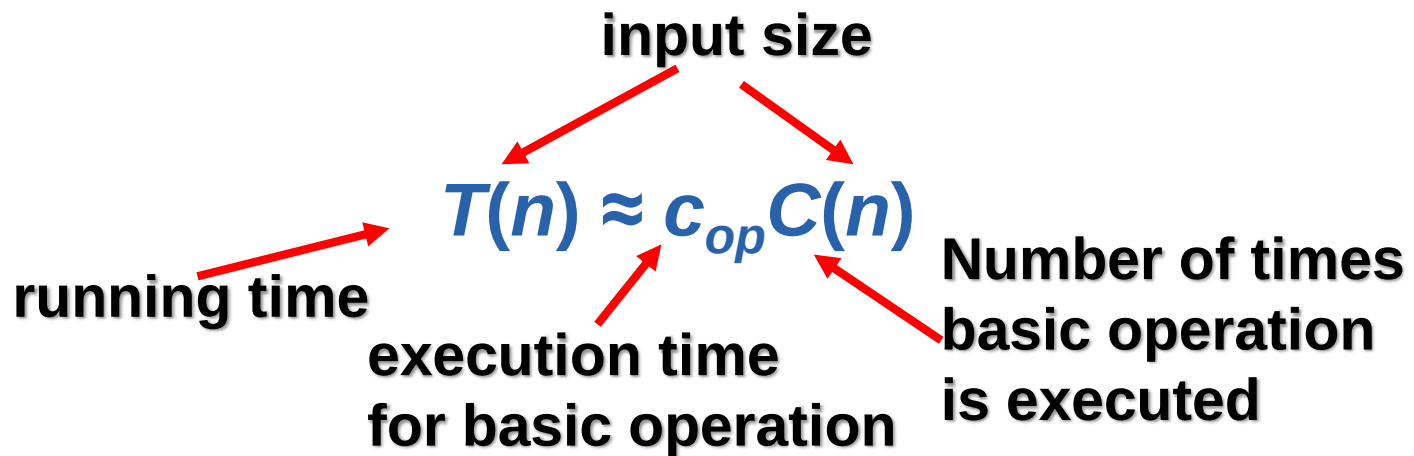
=  $\theta(n)$



# THEORETICAL ANALYSIS OF TIME EFFICIENCY

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size

Basic operation: the operation that contributes most towards the running time of the algorithm



# GENERAL PLAN

How to find  $T(n) \approx c_{op}C(n)$ ??

- Decide on a parameter indicating an **input's size**
- Does  $C(n)$  depends **only on  $n$**  or does it also **depend on type** of input?
- Identify the **basic operation**
- **Set up sum** expressing the # of times basic operation is executed.
- **Find closed form for the sum or at least establish its order of growth**

# ANALYSIS OF NON-RECURSIVE EXAMPLES

**ALGORITHM** MaxElement( A [0..n-1] )

**Times of execution**

maxval  $\leftarrow$  A[0]

**for** i  $\leftarrow$  1 **to** n-1 **do**

**if** A[i] > maxval

maxval  $\leftarrow$  A[i]

**return** maxval

**Input size:** n

**Basic operation:** > or  $\leftarrow$

**Then:**

$$C(n) = \sum_{i=1}^{n-1} 1$$

# ANALYSIS OF NONRECURSIVE (CONTD.)

## Times of execution

...ments(A [0..n-1] )

//Determines whether all elements are distinct

**for** i <- 0 **to** n-2 **do**

**for** j <- i+1 **to** n-1 **do**

**if** A[i] = A[j]

**return false**

**return true**

**Input size:** n

**Basic operation:** A[i] = A[j]

$$\sum_{i=0}^{n-2}$$

$$\sum_{j=i+1}^{n-1}$$

# ANALYSIS OF NONRECURSIVE (CONTD.)

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

# ANALYSIS OF NONRECURSIVE (CONTD.)

**ALGORITHM** MatrixMultiplication(A[0..n-1, 0..n-1],  
B[0..n-1, 0..n-1])

**for** i <- 0 **to** n-1 **do**

**for** j <- 0 **to** n-1 **do**

C[i, j] = 0.0

**for** k <- 0 **to** n-1 **do**

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3.$$

**Basic operation: \***

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

# ANALYSIS OF NONRECURSIVE (CONTD.)

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3.$$

**Input size:  $n$**

**Basic operation:  $\times$**

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

If we took into account the time spent in the additions too

$$T(n) \approx c_m M(n) + c_a A(n) = c_m n^3 + c_a n^3 = (c_m + c_a) n^3,$$

$$\begin{aligned} T(n) &\approx c_m M(n) + c_a A(n) \\ &= c_m n^3 + c_a n^3 \\ &= (c_m + c_a) n^3 \end{aligned}$$

# ANALYSIS OF NONRECURSIVE (CONTD.)

**Algorithm** linearSearch ( $A[0..n-1]$ , key)

for ( $i = 0$  to  $n-1$ )

if ( $A[i] == \text{key}$ )

return  $i$ ;

return  $-1$ ;

**Input size:**  $n$

**Basic operation:**  $==$

**When  $i = 0 \rightarrow A[i] == \text{Key}$**

**Best case:** **Key not found**

$n-1$   
 $\sqrt{\quad}$

**Worst case:**

$i=0$

**Then**  $C(n) \in \mathbf{O(n)}$



# ANALYSIS OF NONRECURSIVE (CONTD.)

**ALGORITHM** Binary(n)

// Output: # of digits in b

// representation of n

count  $\leftarrow$  1

while  $n > 1$  do

    count  $\leftarrow$  count+1

$n \leftarrow \left\lfloor \frac{n}{2} \right\rfloor$

return count

→ After m iteration

$$2^m \leq n \leq 2^{m+1}$$

By taking  $\log_2$

$$m \leq \lg n \leq m+1$$

Then:

$$\underline{C(n) \in \Theta(\lg n)}$$

# ALGORITHMS TECHNIQUES

- ❑ Brute force (ex. Sequential search)
- ❑ Decrease and Conquer (ex. Insertion Sort)
- ❑ Divide and Conquer (ex. Merge – Quick Sort)
- ❑ Transform and Conquer (ex. Heap Sort)
- ❑ Dynamic Programming
- ❑ Greedy Technique.

# DECREASE AND CONQUER

- **Decrease or reduce** problem instance to smaller instance of the same problem and extend solution.
- **Conquer** the problem by solving smaller instance of the problem.



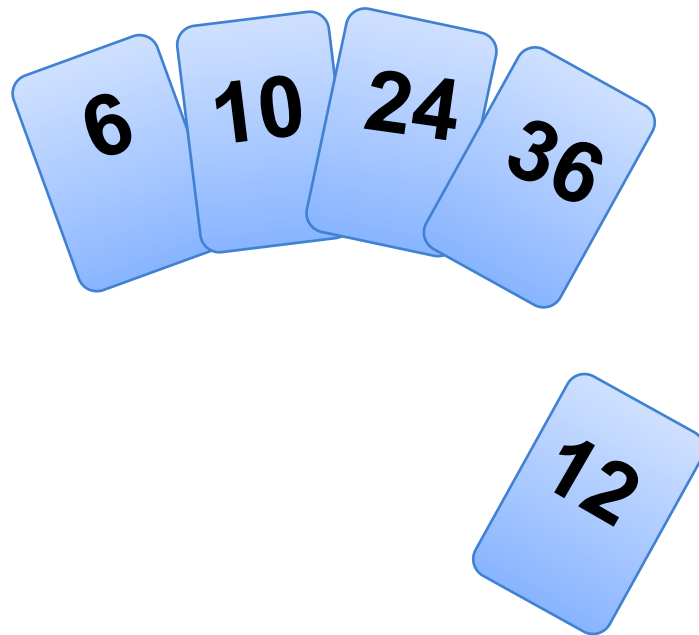
# INSERTION SORT

# INSERTION SORT

**Idea:** like sorting a hand of playing cards

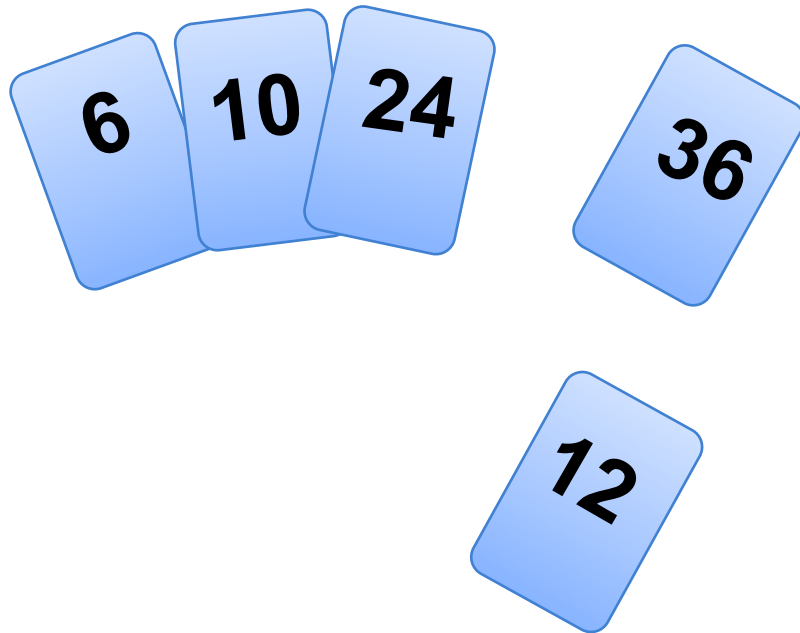
- Start with an empty left hand and the cards facing down on the table.
- Remove one card at a time from the table, and insert it into the correct position in the left hand
  - compare it with each of the cards already in the hand, from right to left
- The cards held in the left hand are sorted
  - these cards were originally the top cards of the pile on the table

# INSERTION SORT

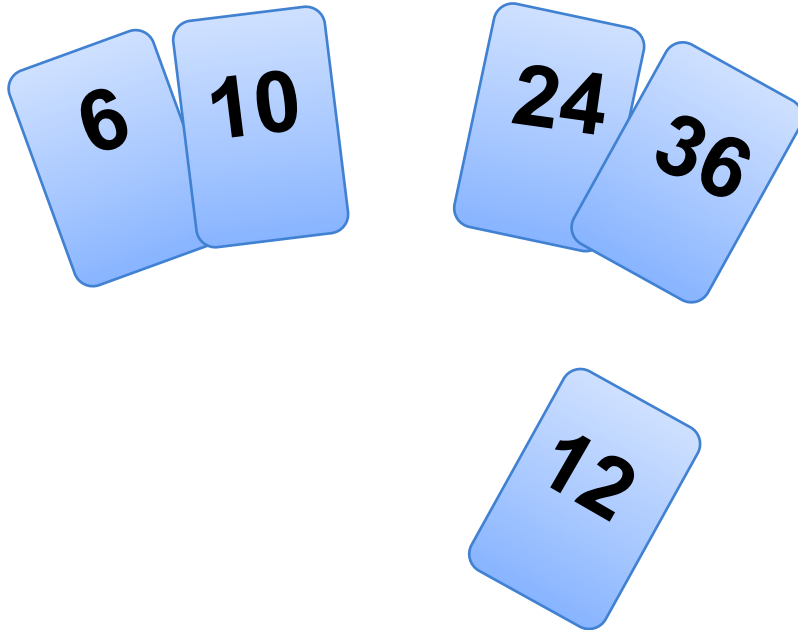


To insert 12, we need to make room for it by moving first 36 and then 24.

# INSERTION SORT



# INSERTION SORT





# INSERTION SORT

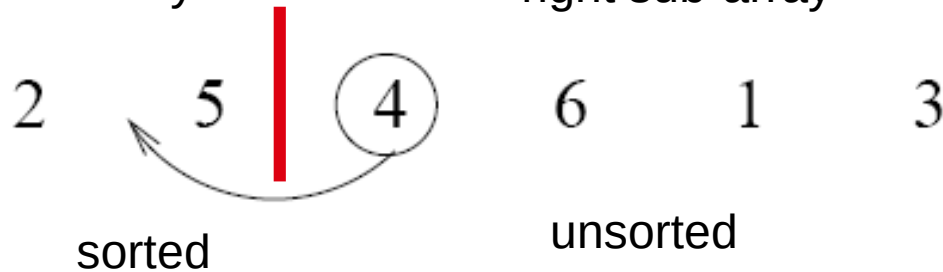
input array

5    2    4    6    1    3

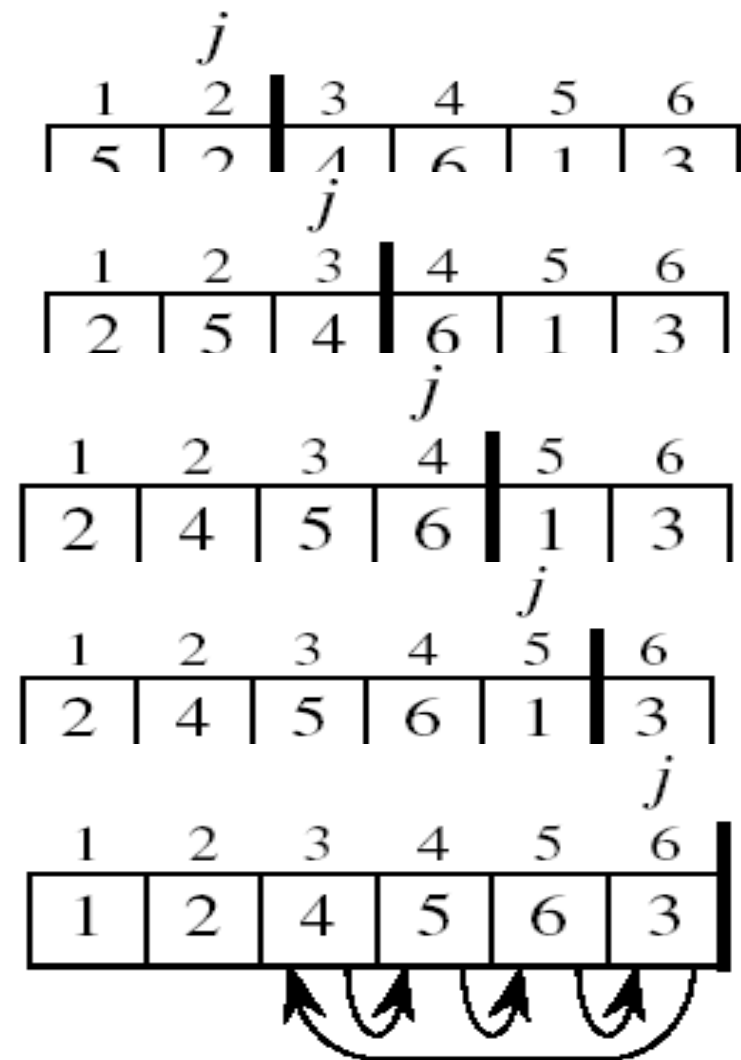
at each iteration, the array is divided in two sub-arrays:

left sub-array

right sub-array



# INSERTION SORT



# INSERTION-SORT

| 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     |
|-------|-------|-------|-------|-------|-------|-------|-------|
| $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | $a_7$ | $a_8$ |

key

INSERTION-SORT(A)

for  $j \leftarrow 2$  to  $n$

$key \leftarrow A[j]$

    //Insert  $A[j]$  into the sorted sequence  $A[1 \dots j-1]$

$i \leftarrow j - 1$

    while  $i > 0$  and  $A[i] > key$

$A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow key$

# ANALYSIS OF INSERTION SORT

## INSERTION-SORT(A)

**for**  $j \leftarrow 2$  **to**  $n$  **do**

$\text{key} \leftarrow A[j]$

$i \leftarrow j - 1$

**while**  $i > 0$  and  $A[i] > \text{key}$

**do**  $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow \text{key}$

The for loop main operation

Assume  $t_j$  is # of times the while statement is executed at iteration  $j$

# BEST CASE ANALYSIS

The array is already sorted

- $A[i] \leq \text{key}$  upon the first

time the **while** loop test is run

- $t_j = 1$

**Then:** 
$$T(n) = \sum_{j=2}^n 1$$

**while**  $i > 0$  and  $A[i] > \text{key}$

$A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

**Best case of  $T(n) \in \Theta(n)$**

# WORST CASE ANALYSIS

The array is reversed

- Always  $A[i] > \text{key}$

**while**  $i > 0$  and  $A[i] > \text{key}$

$A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

- Have to compare key with all elements to the left of the  $j$ -th position  $\Rightarrow$  compare with  $j-1$  elements

- $\Rightarrow t_j = j$

**Then:** 
$$T(n) = \sum_{j=2}^n j$$

# WORST CASE ANALYSIS

Using  $\sum_{j=1}^n j = \frac{n(n+1)}{2}$

$$T(n) = \sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

**Worst case of  $T(n) \in \Theta(n^2)$**

# INSERTION SORT - SUMMARY

## Advantages

- Good running time for “almost sorted” arrays  $\Theta(n)$

## Disadvantages

- $\Theta(n^2)$  running time in **worst** and **average** case





# **DIVIDE AND CONQUER ALGORITHM DESIGN STRATEGY**

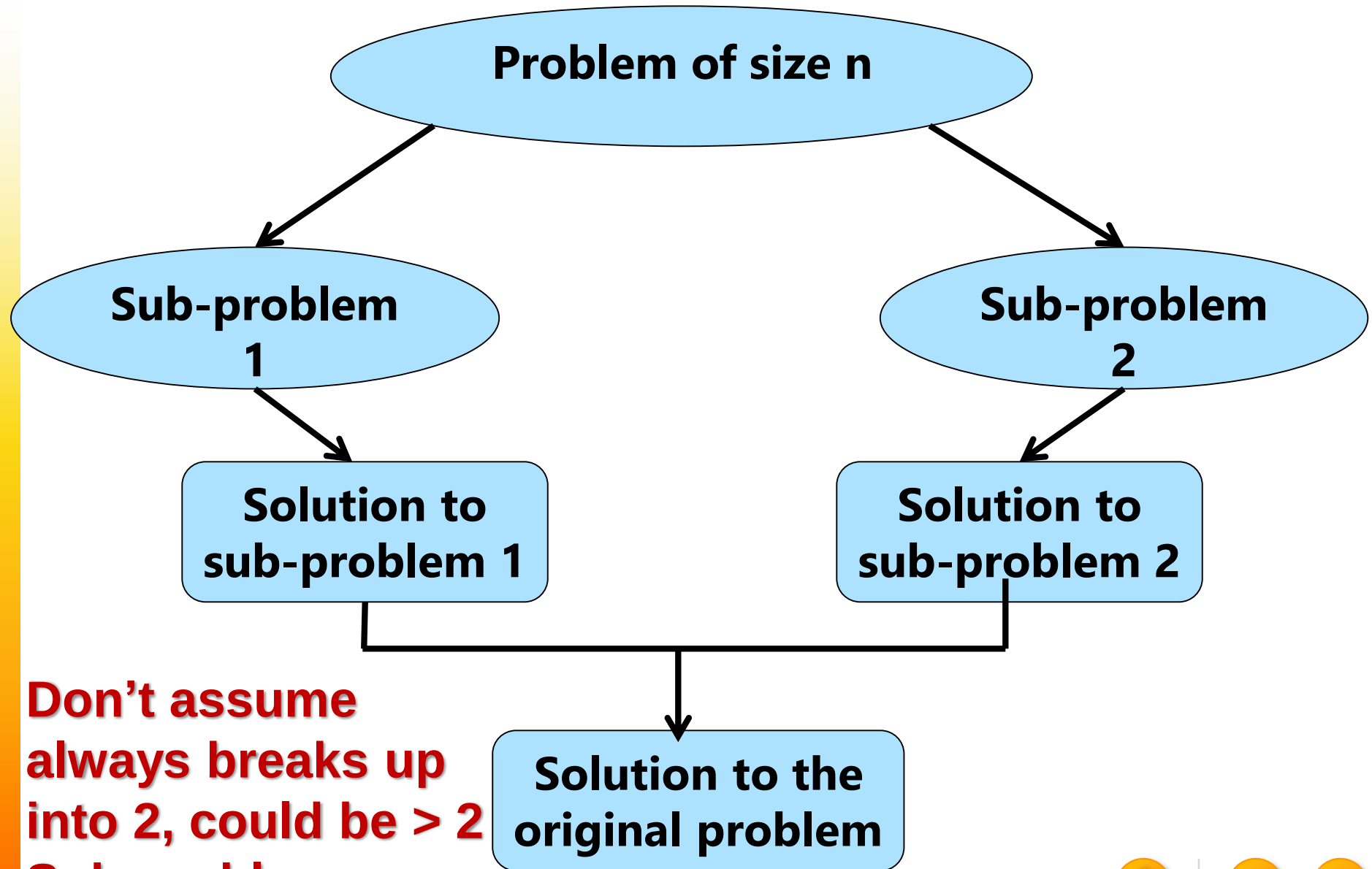
# DIVIDE AND CONQUER

Three steps are involved:

- Divide the problem into several subproblems, perhaps **of equal size**
- Subproblems are solved, typically **recursively**
- The solutions to the subproblems are combined to get a solution to the original problem



# DIVIDE AND CONQUER (CONTD.)



**Don't assume  
always breaks up  
into 2, could be  $> 2$   
Sub-problems**



# DIVIDE AND CONQUER (CONTD.)

**Example 1:** Divide array into two halves, recursively sort left and right halves, then merge two halves → **Merge sort**

**Example 2:** Partition array into items that are “small” and items that are “large”, then recursively sort the two sets → **Quicksort**

**Example 3:** Partition array into two halves from ‘mid’, search into the left half if the search key is smaller than ‘mid’ or search into the right half otherwise → **Binary search**



A decorative yellow curved shape, resembling a stylized wave or a partial arc, is positioned at the top of the slide, curving from the left edge towards the right.

# MERGE SORT

# MERGE SORT

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 8 | 2 | 9 | 4 | 5 | 3 | 1 | 6 |
|---|---|---|---|---|---|---|---|

**Sorting Problem:** Sort a sequence of  $n$  elements into non-decreasing order.

***Divide:*** Divide the  $n$ -element sequence to be sorted into two subsequences of  $n/2$  elements each

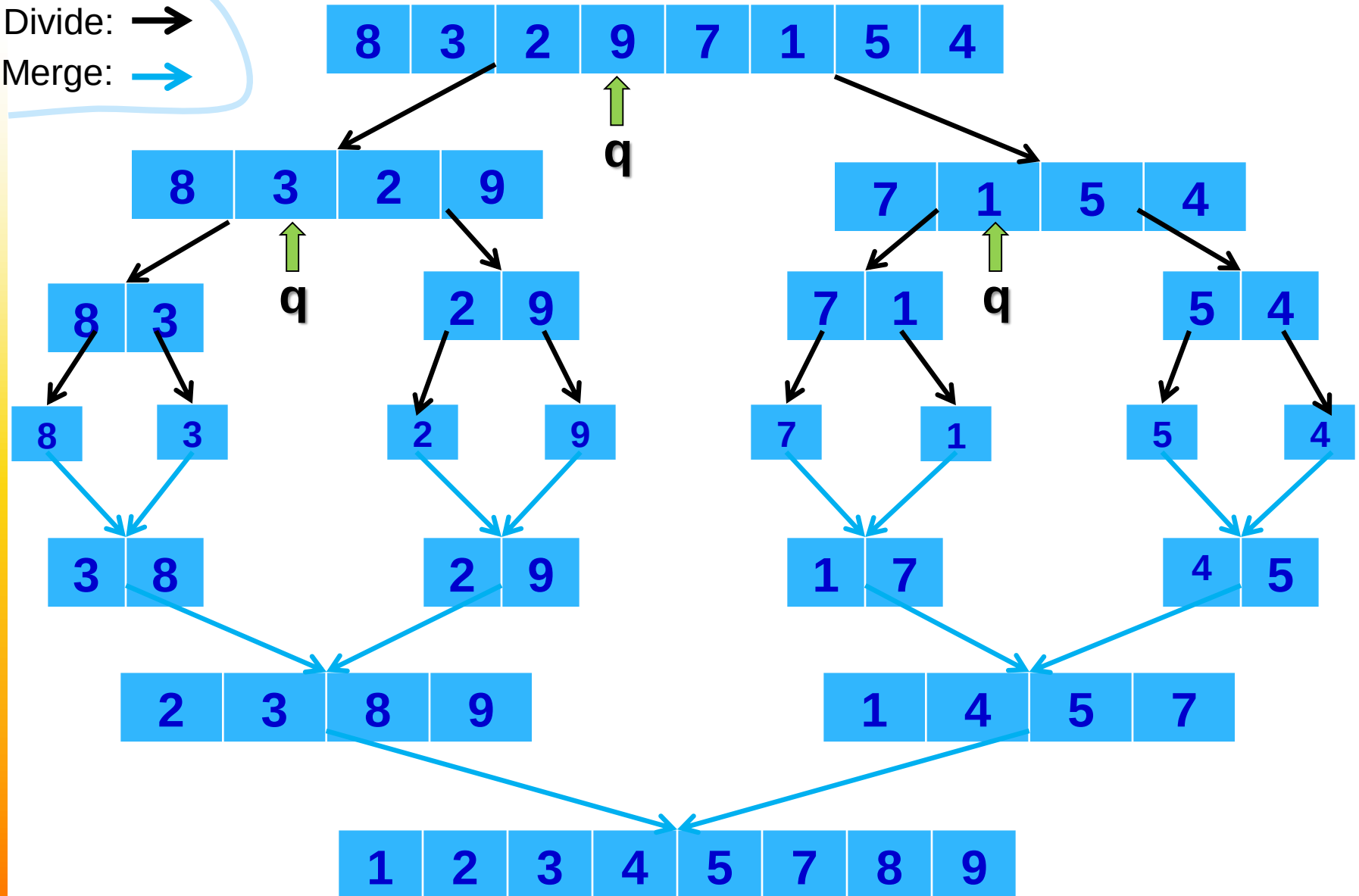
***Conquer:*** Sort the two subsequences recursively using merge sort.

***Combine:*** Merge the two sorted subsequences to produce the sorted answer.



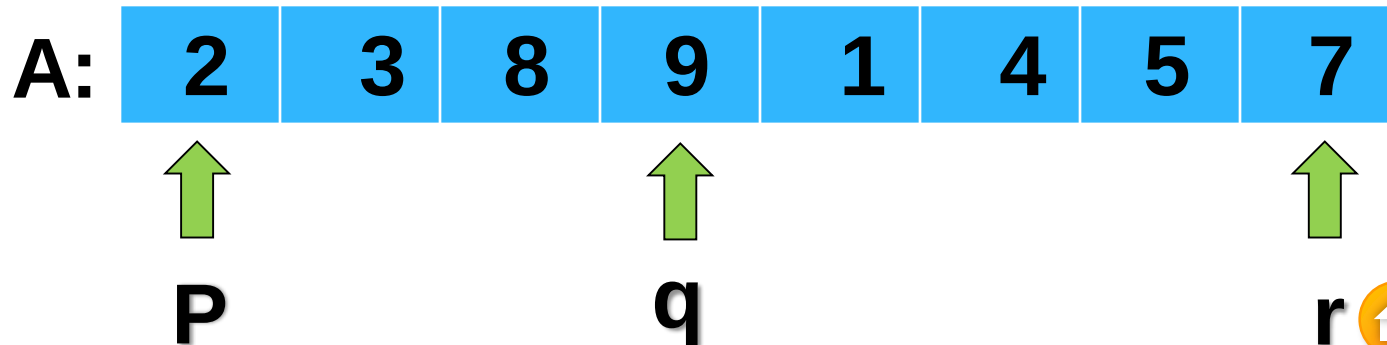
# MERGE SORT (CONTD.)

Divide: →  
Merge: →



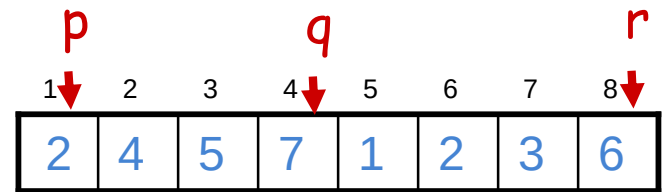
# MERGE SORT(CONTD.)

```
Algorithm Merge-Sort (A, p, r) :  
  if p < r then  
    q ← ⌊(p+r) / 2⌋  
    Merge-Sort (A, p, q)  
    Merge-Sort (A, q+1, r)  
  Merge (A, p, q, r)
```



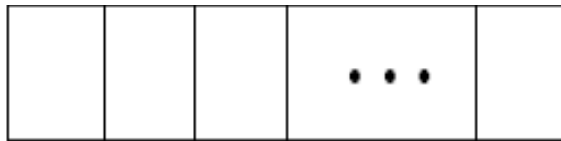


# MERGING IDEA

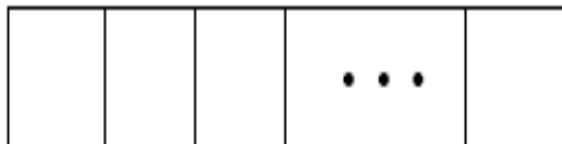


- Copy each of the two sorted sub-sets into two temp sets
- Choose the smaller of the two top elements of the two temp sets
- Remove it and place it in the output set
- Repeat the process until one set is empty
- Take the remaining and place them onto the output set

$A1 \leftarrow A[p, q]$

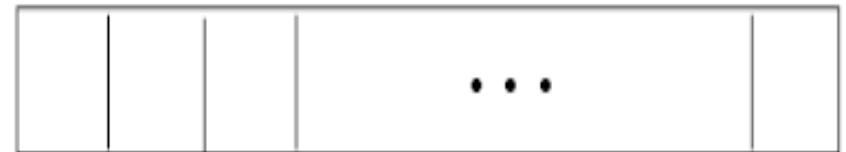


$A2 \leftarrow A[q+1, r]$



choose the smaller  
element from the subarrays

$A[p, r]$

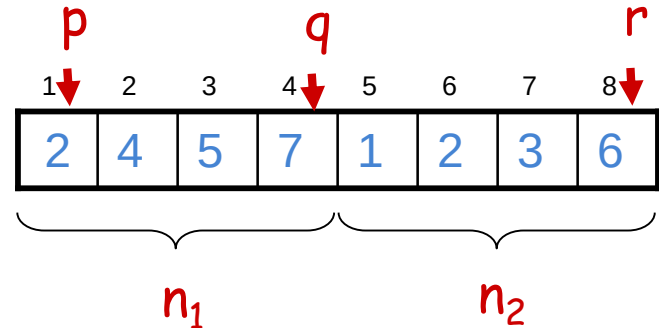


# PSEUDOCODE OF THE MERGE PROCEDURE

MERGE ( $A, p, q, r$ )

//Compute  $n_1$  and  $n_2$

1.  $n_1 \leftarrow q - p + 1$
2.  $n_2 \leftarrow r - q$



**L**

|   |   |   |   |          |
|---|---|---|---|----------|
| 2 | 4 | 5 | 7 | $\infty$ |
|---|---|---|---|----------|

**R**

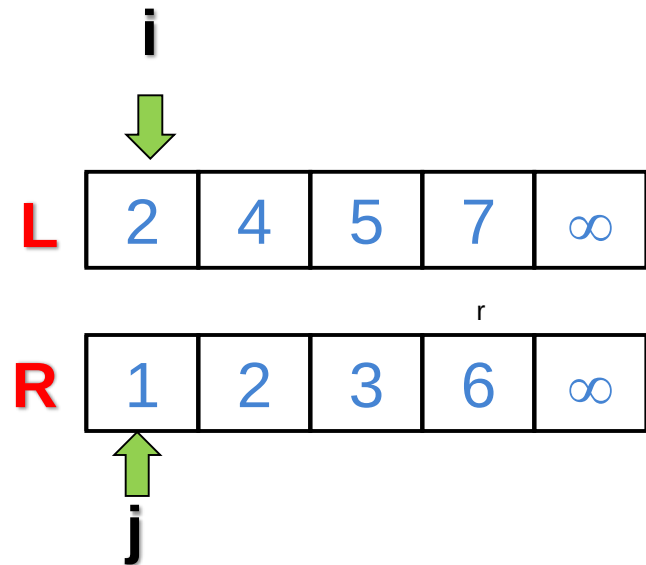
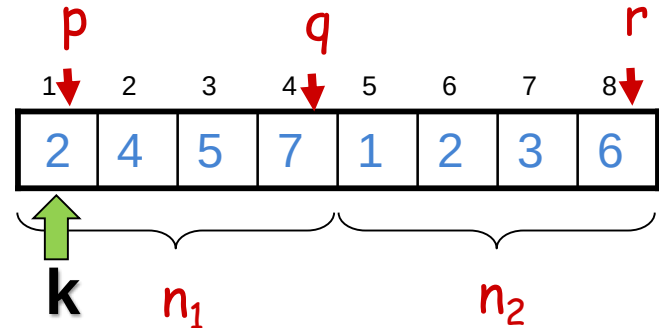
|   |   |   |   |          |
|---|---|---|---|----------|
| 1 | 2 | 3 | 6 | $\infty$ |
|---|---|---|---|----------|



# PSEUDOCODE OF THE MERGE PROCEDURE

MERGE ( $A, p, q, r$ )

...



# ANALYSIS OF MERGE SORT

**ALGORITHM** Mergesort( $A, p, r$ )

IF  $p < r$

THEN  $q = \text{FLOOR}[(p + r)/2]$

MERGE\_Sort ( $A, p, q$ )                       $T(n/2)$

MERGE\_Sort ( $A, q + 1, r$ )                   $T(n/2)$

MERGE ( $A, p, q, r$ )                           $\theta(n)$

---

$$T(n) = 2T(n/2) + \theta(n)$$

recurrence relation



A decorative yellow curved shape, resembling a stylized wave or a partial arc, is positioned at the top of the slide, spanning across the width of the page.

# **BINARY SEARCH**

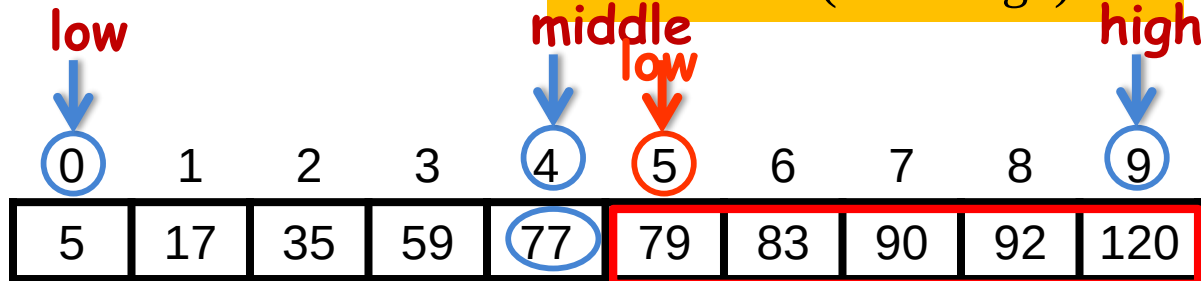
# BINARY SEARCH

**low** → low index in range  
**high** → high index in range  
**middle** =  $\lfloor (low + high) / 2 \rfloor$

Initially:

low=0,

high=size-1

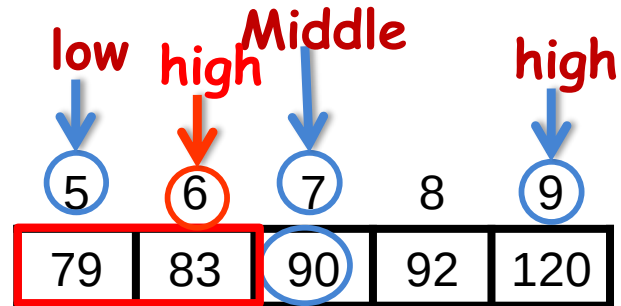


**Key**

79

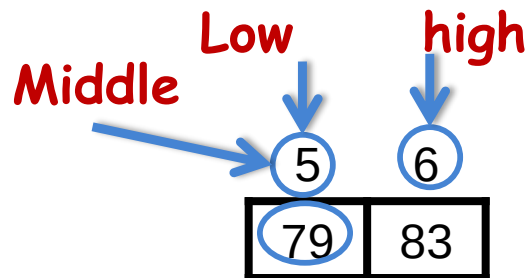
key > a [middle]

low = middle + 1



Key < a [middle]

high = middle - 1



Key == a [middle]

Return Middle

In our example, it returns the index 5 after only 3 comparisons

# BINARY-SEARCH: RECURRENCE

For an *ordered array A*, finds if *x* is in the *array A[lo...hi]*

**BINARY-SEARCH (A, lo, hi, x)**

**if** (lo > hi)

**return** FALSE

mid  $\leftarrow \lfloor (\text{lo} + \text{hi}) / 2 \rfloor$

**if** x = A[mid]

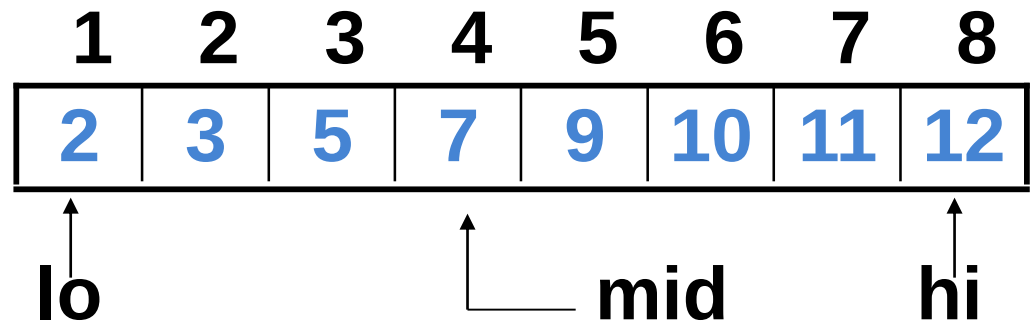
**return** TRUE

**if** ( x < A[mid] )

    BINARY-SEARCH (A, lo, mid-1, x)

**if** ( x > A[mid] )

    BINARY-SEARCH (A, mid+1, hi, x)



# BINARY-SEARCH: RECURRENCE

BINARY-SEARCH (A, lo, hi, x)

if (lo > hi)

return

Because of the **IF** condition only one of the two calls will be executed

constant time:  $c_1$

constant time:  $c_2$

if

return TRUE

constant time:  $c_3$

if ( x < A[mid] )

BINARY-SEARCH (A, lo, mid-1, x)

same problem of size  $n/2$

if ( x > A[mid] )

BINARY-SEARCH (A, mid+1, hi, x)

same problem of size  $n/2$

$$T(n) = c + T(n/2)$$



# **APPENDIX: USEFUL MATHEMATICAL FORMULAS FOR THE ANALYSIS OF ALGORITHMS**

# APPENDIX: IMPORTANT SUMMATION FORMULAS

$$1. \quad \sum_{i=l}^u 1 = 1 + 1 + \cdots + 1 = u - l + 1$$

*(l,u are integer limits,  $l \leq u$ );  $\sum_{i=1}^n 1 = n$*

$$2. \quad \sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2$$

$$3. \quad \sum_{i=1}^n i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{1}{3}n^3$$

$$4. \quad \sum_{i=1}^n i^k = 1^k + 2^k + \cdots + n^k \approx \frac{1}{k+1}n^{k+1}$$

$$5. \quad \sum_{i=0}^n a^i = 1 + a + \cdots + a^n = \frac{a^{n+1}-1}{a-1} \quad (a \neq 1) = \sum_{i=0}^n 2^i = 2^{n+1}-1$$

# APPENDIX: IMPORTANT SUMMATION FORMULAS

$$6. \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \ln n + \gamma$$

*, where  $\gamma \approx 0.5772$  (Euler's Constant)*

$$7. \sum_{i=1}^n i2^i = 1 * 2 + 2 * 2^2 + \dots + n2^n = (n-1)2^{n+1} + 2$$

$$8. \sum_{i=1}^n ia^i = 1 * 2 + 2 * 2^2 + \dots + a \frac{d}{da} \left( \frac{a^{n+1}-1}{a-1} \right)$$

$$9. \sum_{i=1}^n \lg i \approx n \lg n$$

$$10. \sum_{i=l}^u ca_i = c \sum_{i=l}^u a_i$$

$$11. \sum_{i=l}^u (a_i \pm b_i) = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i$$

# APPENDIX: PROPERTIES OF LOGARITHMS

1.  $\log_a 1 = 0$

2.  $\log_a a = 1$

3.  $\log_b(m^n) = n \cdot \log_b(m)$

4.  $\log_b(mn) = \log_b(m) + \log_b(n)$

5.  $\log_b(m/n) = \log_b(m) - \log_b(n)$

6.  $a^{\log_b m} = m^{\log_b a}$

7.  $\log_a m = \frac{\log_b m}{\log_b a} = \log_a b \log_b m$